

Fine-Tuning a Local Language Model for Structured Resume-Job Fit Analysis

Nicholas Fairhart

May 2026

University of Hawaii at Manoa

ICS 605 — Final Project

The Problem

Job searching is a communication problem.

- Candidates may have the right skills but resumes don't "speak the language" of job descriptions
- Recruiters review hundreds of applications, often without domain expertise
- Especially hard for **career-returners**: legitimate experience, employment gap stigma

Existing tools fall short:

- General-purpose LLM chat gives isolated line feedback
- No structured output → hard to act on
- Requires API access / ongoing cost

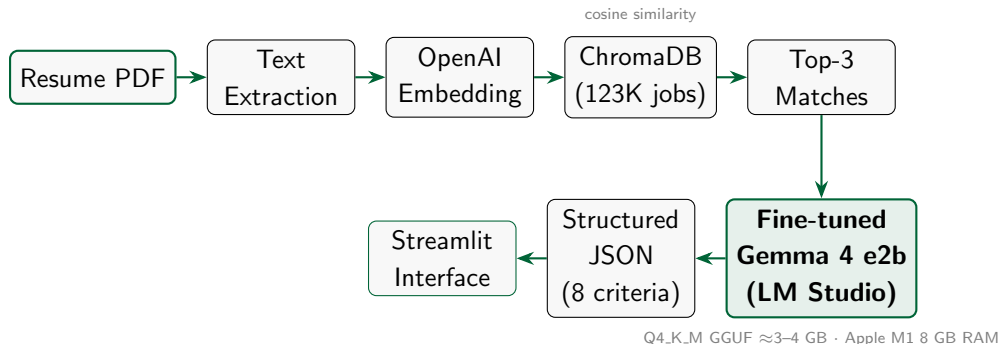
Goal

Build a **structured, actionable** fit analyzer that runs **entirely on consumer hardware**—no cloud, no recurring cost.

Eight output criteria

Match score · Experience level fit · Matching strengths · Skill gaps · ATS keyword coverage · Resume improvements · Recommended activities

System Overview



Fully local inference after training — **no API cost**, no internet required at runtime

Two public Kaggle datasets

- **Resumes:** 2,483 across 25 categories (healthcare, engineering, finance, ...)
- **Jobs:** 123,849 LinkedIn postings with title, company, experience level, description

Pair construction

- 100 resumes sampled per category (24 categories)
- Per resume: **top-3 semantic matches** via ChromaDB + **2 random** postings
- Random pairs add score variance; stopped at **10,000 pairs**

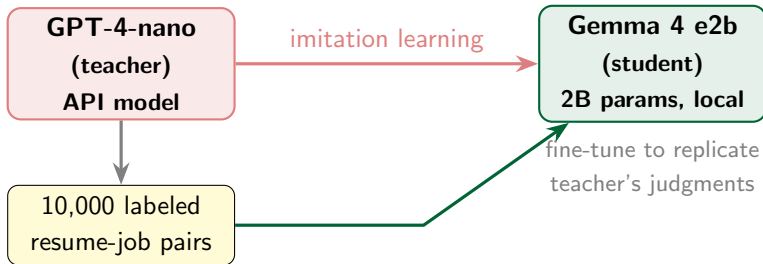
Teacher labeling — GPT-4-nano

- Structured output schema: 8 criteria per pair
- ≈ 17 M tokens processed
- Total cost: **\$4.68**
- Goal: *consistent* scoring behavior, not perfect ground truth

Final split

Train	9,000 pairs
Validation	997 pairs
Total	$\sim 10,000$ pairs

Method — Knowledge Distillation



Why distillation?

- Teacher's structured outputs serve as pseudo-labels
- No human annotation needed
- Student model runs locally at inference

Output schema (JSON)

- `match_score`: 0–100
- `experience_level`: under/match/over
- `matching_strengths`, `skill_gaps`
- `ats_keywords`, `resume_improvements`

Method — LoRA Fine-tuning & Deployment

LoRA configuration

- Target modules: q_proj, v_proj, o_proj
- Rank $r = 8$, $\alpha = 16$
- **3.7M trainable** / 5.1B total = **0.07%**
- Loss only on model response
(train_on_responses_only)

Training hyperparameters

- $LR = 1 \times 10^{-5}$, cosine schedule, 100 warmup steps
- Dropout 0.1, weight decay 0.05
- 3 epochs, batch size 32, early stopping
- ≈ 1 hour on Google Colab A100 (40 GB)

Deployment pipeline

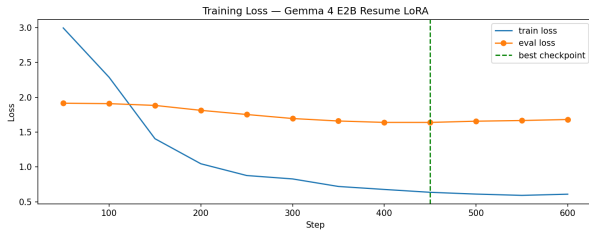
1. Train in BF16 on Colab A100
2. Merge LoRA adapter with base weights
3. Quantize to **Q4_K_M GGUF** via llama.cpp
4. ≈ 10 GB BF16 $\rightarrow \approx 3\text{--}4$ GB GGUF
5. Serve via **LM Studio**
(OpenAI-compatible API)
6. Streamlit calls local endpoint

Key design decisions

Fewer target modules + lower LR fixed overfitting.

Response-only masking fixed

Training Loss



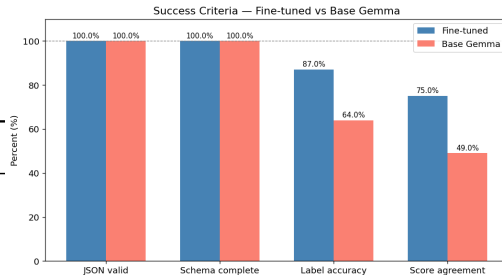
- Both train and val loss decrease steadily
- Best checkpoint restored at step 450 (early stopping)
- **Persistent train/val gap:** model learned the task but did not fully generalize
- Likely causes: label noise, modest dataset size, BF16 train vs. Q4 eval mismatch

Results

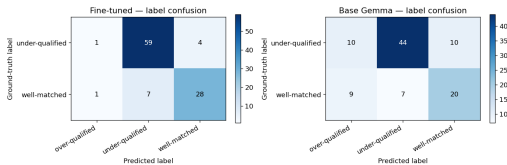
Table 1: Evaluation on 100 held-out examples

Metric	Base	Fine-tuned
Score agreement (± 10 pts)	49%	75%
Experience level accuracy	64%	87%
Match score MAE	13.49	9.11
Over-qualified FP	19	2

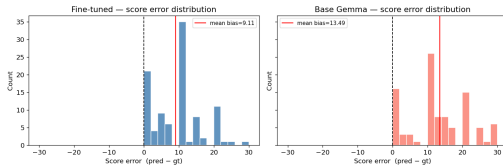
+26 pp score agreement
classification accuracy **+23 pp**



Results — Classification & Score Error



Base model: 19/100 false “over-qualified”
with zero ground-truth over-qualified cases.
Fine-tuned: reduced to 2.



Fine-tuned errors cluster near 0; base model
spreads to ± 30 pts. Both models show
consistent positive bias.

Streamlit app — three pages

1. **Search:** upload resume PDF → retrieve top matching jobs from ChromaDB
2. **Analyze:** select a job → structured 8-criteria feedback from fine-tuned Gemma
3. **History:** review past analyses

Success case:

Software engineer resume matched to a Senior SWE posting — model correctly identifies matching skills, gaps, and actionable improvements.

Failure case / limitations

- **Positive scoring bias:** weak matches score 10–15 pts above teacher label
- **Short/sparse resumes:** JSON may be missing fields or truncated
- **LM Studio must be running:** local server required—no cloud fallback

Strengths & Limitations

What worked

- Full end-to-end pipeline: data → LoRA → quant → local deployment
- Meaningful gains from \$4.68 of teacher labels
- Runs on M1 8 GB RAM — no cloud budget at inference
- Backend-agnostic: swap model without changing UI

Limitations & next steps

- **Persistent positive scoring bias** — both models over-predict
- **Data quantity is the binding constraint** — 9K examples insufficient for full generalization
- **Better data strategy**: synthetic resumes at varying fit levels

Takeaways

1. **Knowledge distillation scales down:**
\$4.68 + 1 hr A100 → **+26 pp** over base model
2. **Data quantity is king:**
train/val gap confirmed it directly
3. **Design for swap-ability:**
configurable backend, no UI changes needed to upgrade
4. **Edge deployment is real:**
Q4 quantization makes 5B-param models viable on commodity hardware

Core result

Score agreement

49% → 75%

Experience level accuracy

64% → 87%

Runs on an Apple M1 with 8 GB RAM.
No API. No cloud. No ongoing cost.

Thank you. Questions?